

Chapter 12

Exception Handling & Text IO

Motivations

When thousands of users log into an **online banking system**, a single unexpected **exception** should not crash the entire system. If it does, transactions fail, and users are locked out.

*How to handle **exceptions** to **prevent catastrophic failures** and **ensure system stability**?*

Motivations

When thousands of users log into an **online banking system**, a single unexpected **exception** should not crash the entire system. If it does, transactions fail, and users are locked out.

*How to handle **exceptions** to **prevent catastrophic failures** and **ensure system stability**?*

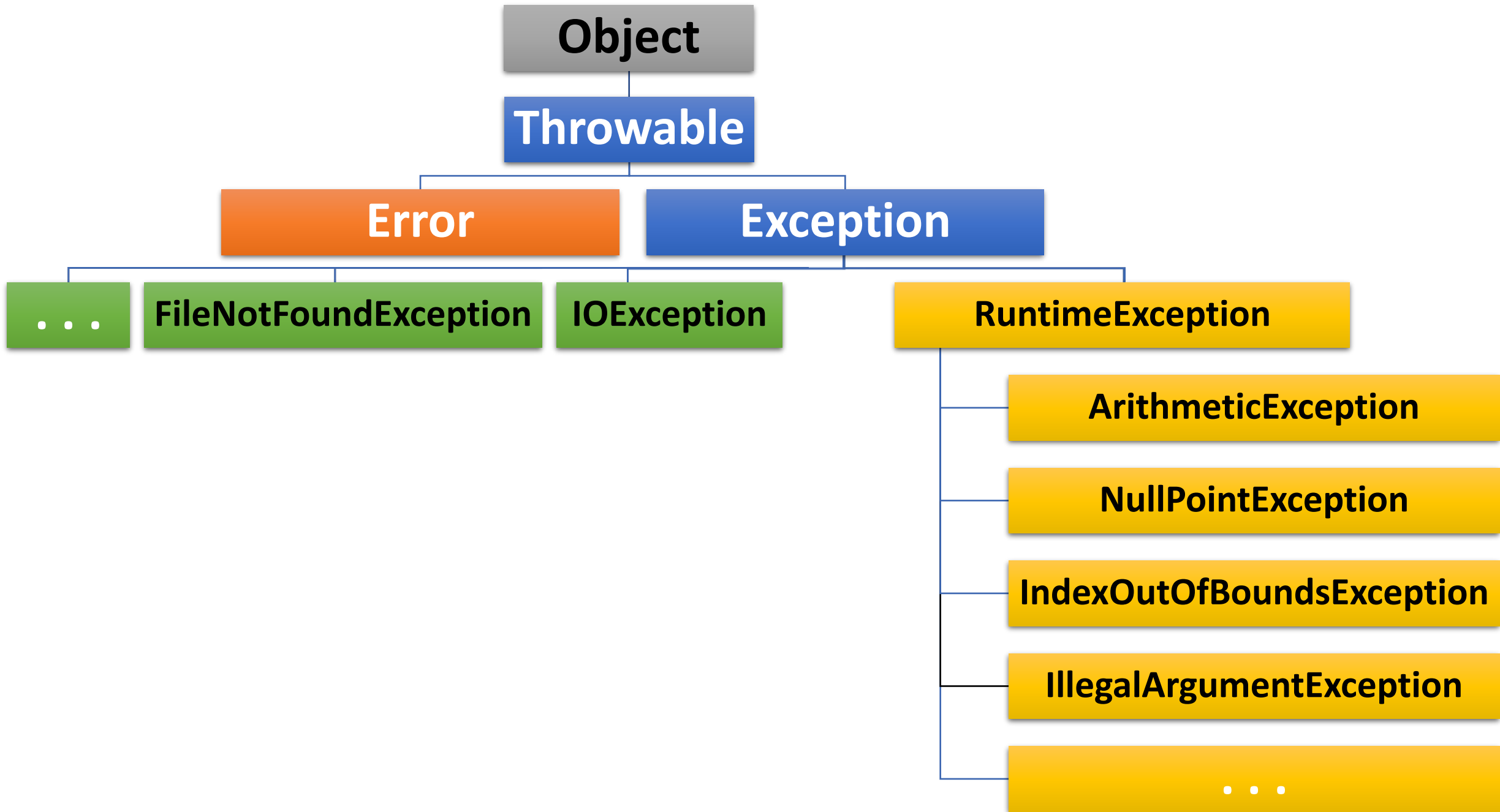


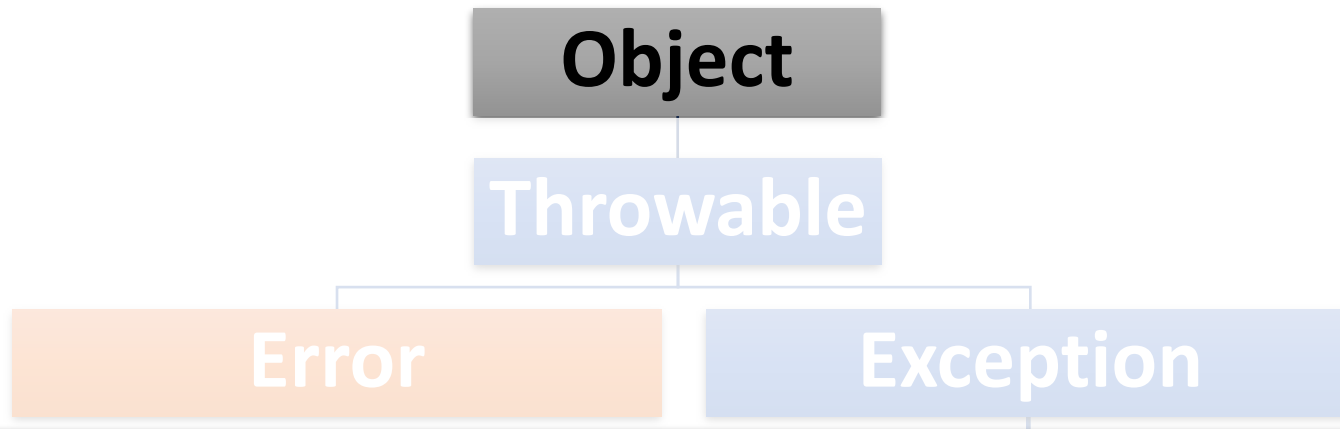
Objectives

- To get an overview of exceptions and exception handling (§12.2)
- To explore the advantages of using exception handling (§12.2)
- To distinguish exception types: **Error** (fatal) vs. **Exception** (nonfatal) and checked vs. unchecked (§12.3)
- To declare exceptions in a method header (§12.4.1)
- To throw exceptions in a method (§12.4.2)
- To write a **try-catch** block to handle exceptions (§12.4.3)
- To explain how an exception is propagated (§12.4.3)
- To obtain information from an exception object (§12.4.4)
- To develop applications with exception handling (§12.4.5)
- To use the **finally** clause in a **try-catch** block (§12.5)
- To use exceptions only for unexpected errors (§12.6)
- To rethrow exceptions in a **catch** block (§12.7)
- To create chained exceptions (§12.8)
- To define custom exception classes (§12.9)

Objectives

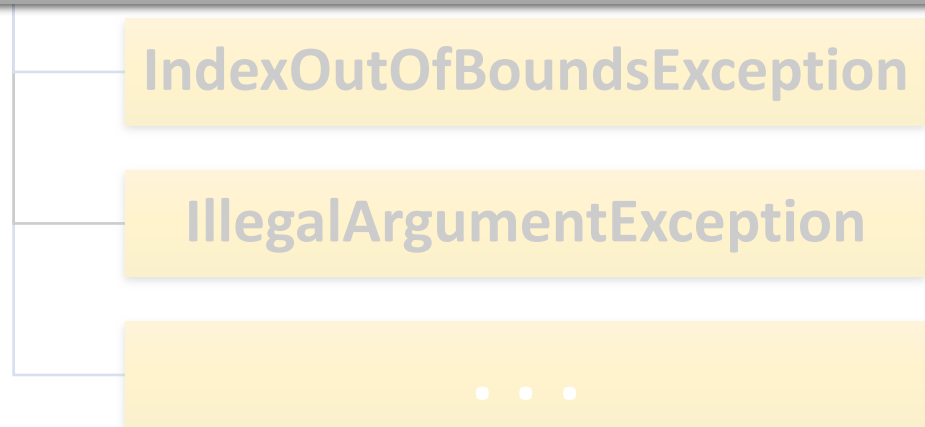
- To discover file/directory properties, to delete and rename files/directories, and to create directories using the **File** class (§12.10)
- To write data to a file using the **PrintWriter** class (§12.11.1)
- To use try-with-resources to ensure that the resources are closed automatically (§12.11.2)
- To read data from a file using the **Scanner** class (§12.11.3)
- To understand how data is read using a **Scanner** (§12.11.4)
- To develop a program that replaces text in a file (§12.11.5)
- To read data from the Web (§12.12)
- To develop a Web crawler (§12.13)

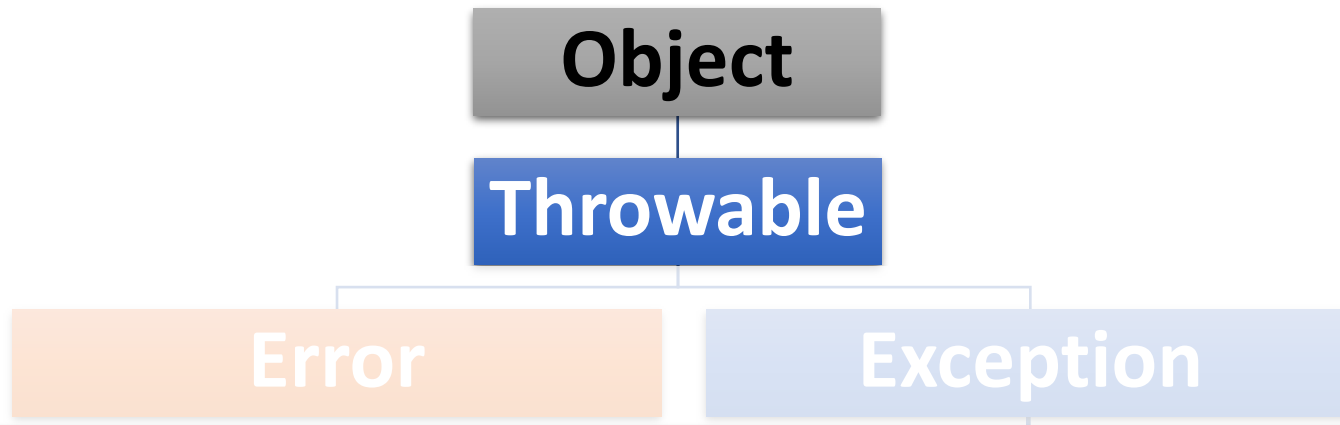




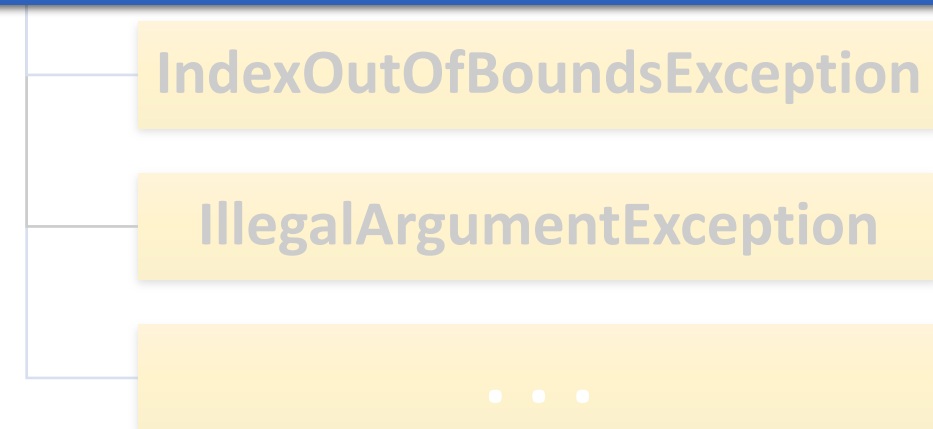
Object class: the top-most class from which every class inherits.
the root of the class hierarchy.

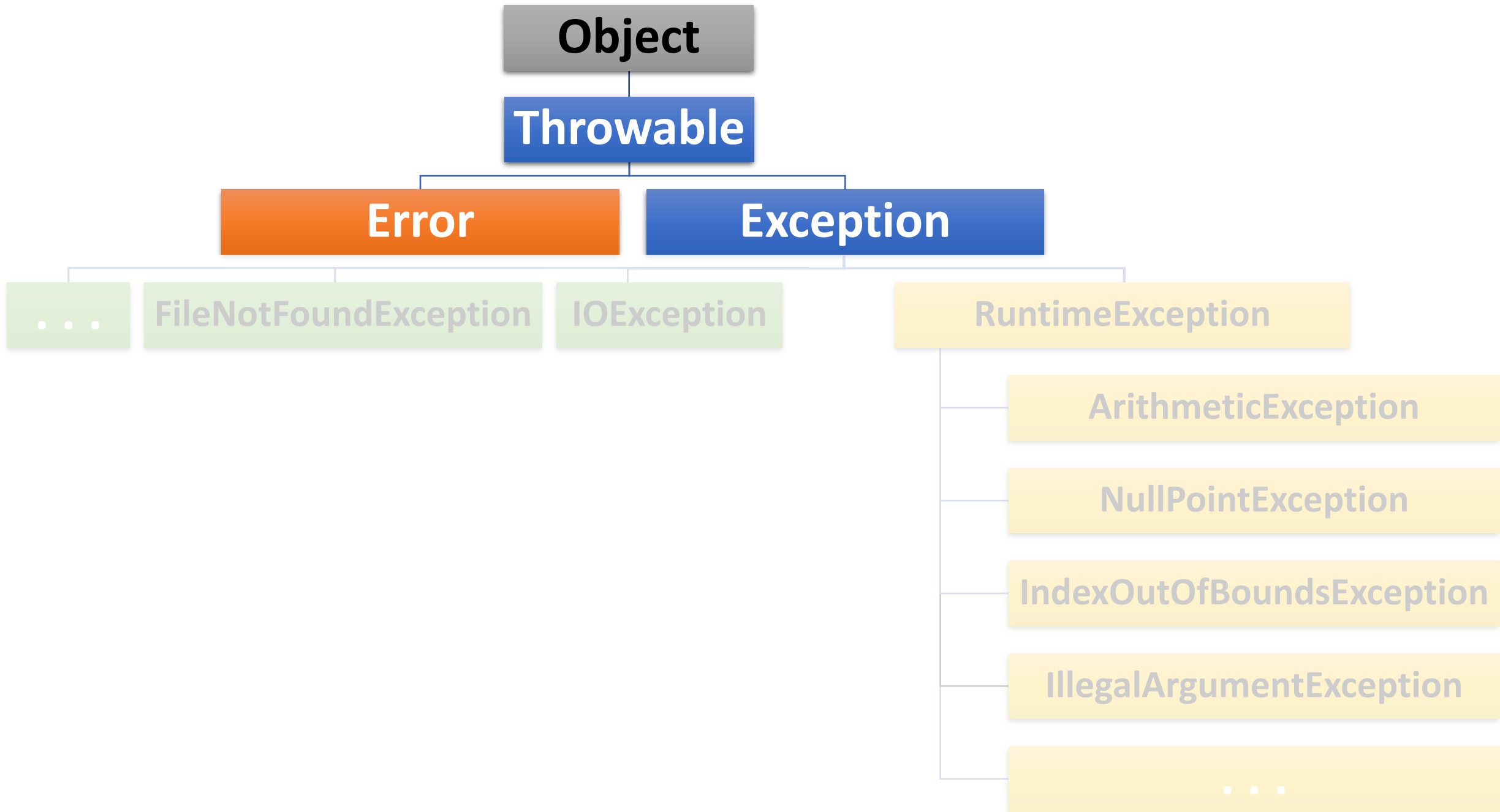
Note: **Object** (class) $\neq$ "object" (instance of a class).

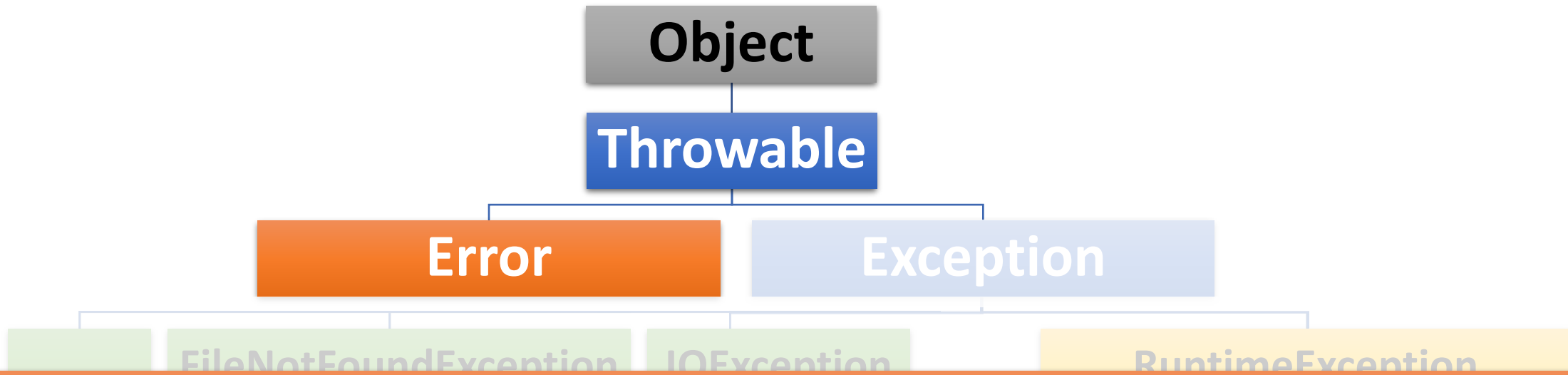




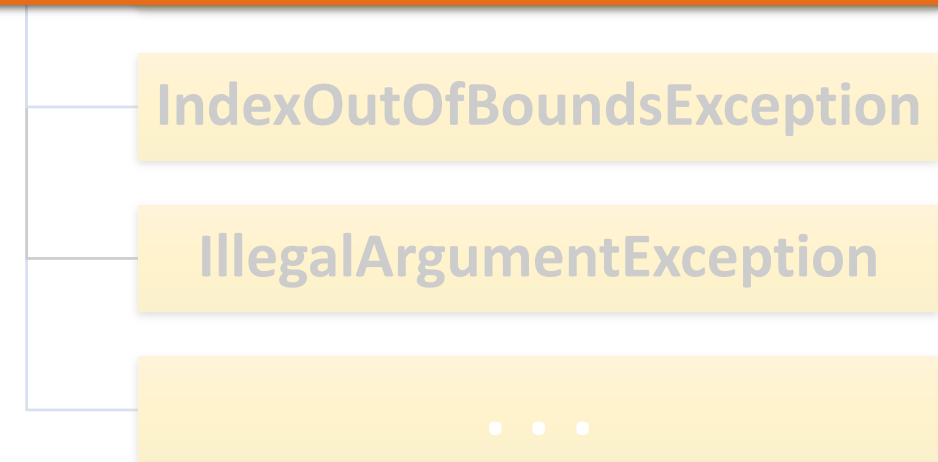
Throwable: the direct parent class of both **Exception** and **Error**.
the base for the entire **exception-handling mechanism**.
all **exception** and **error** in Java are subclasses of Throwable.

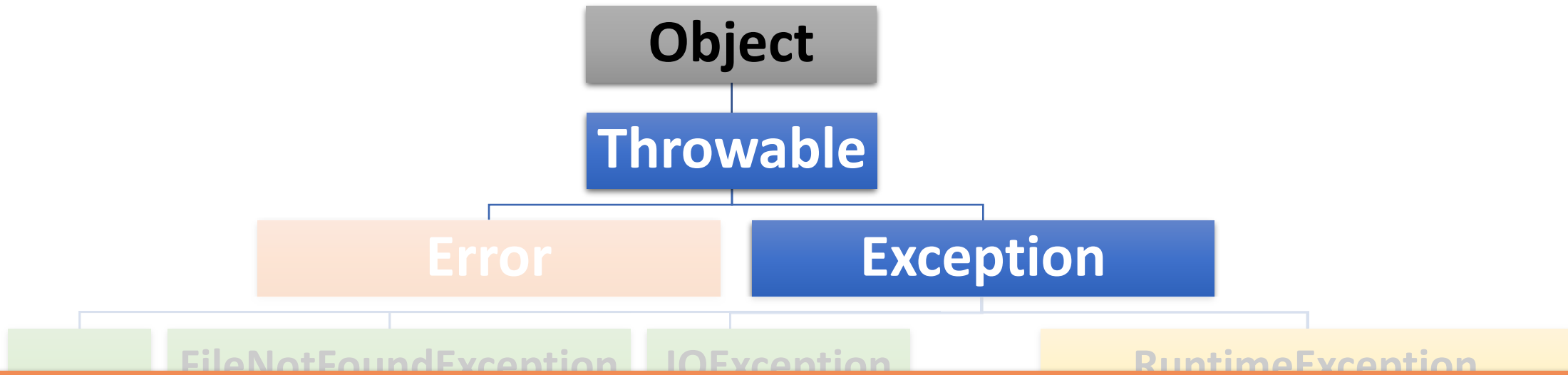






Error: serious **system failures** that cannot be handled, such as **OutOfMemoryError**, which occurs at **runtime** and is considered **system-level runtime error**.

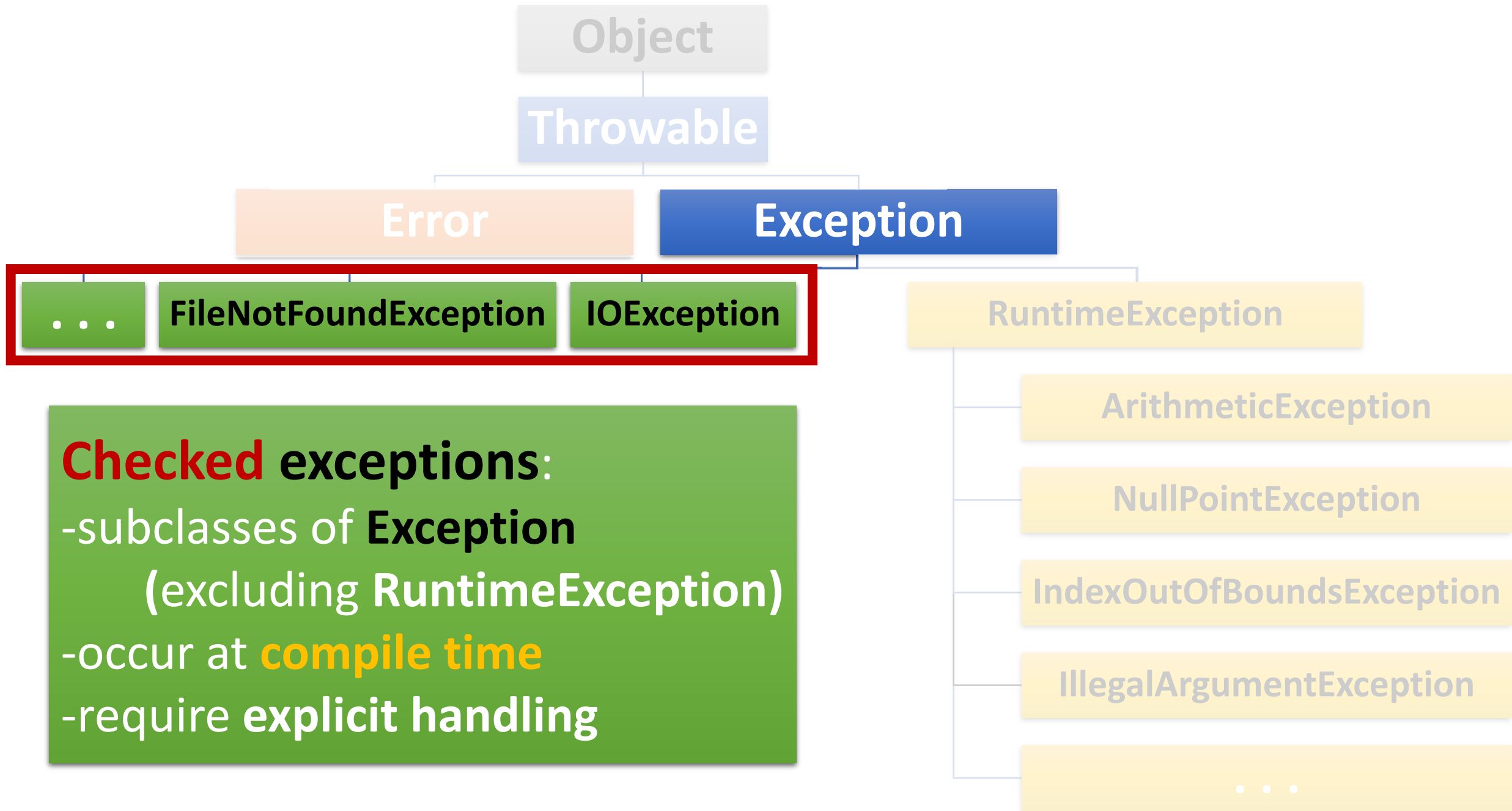




Error: serious **system failures** that cannot be handled, such as **OutOfMemoryError**, which occurs at **runtime** and is considered **system-level runtime error**.

Exception: **recoverable issues** that can be handled by a programmer.
Checked exceptions & **Unchecked** exceptions

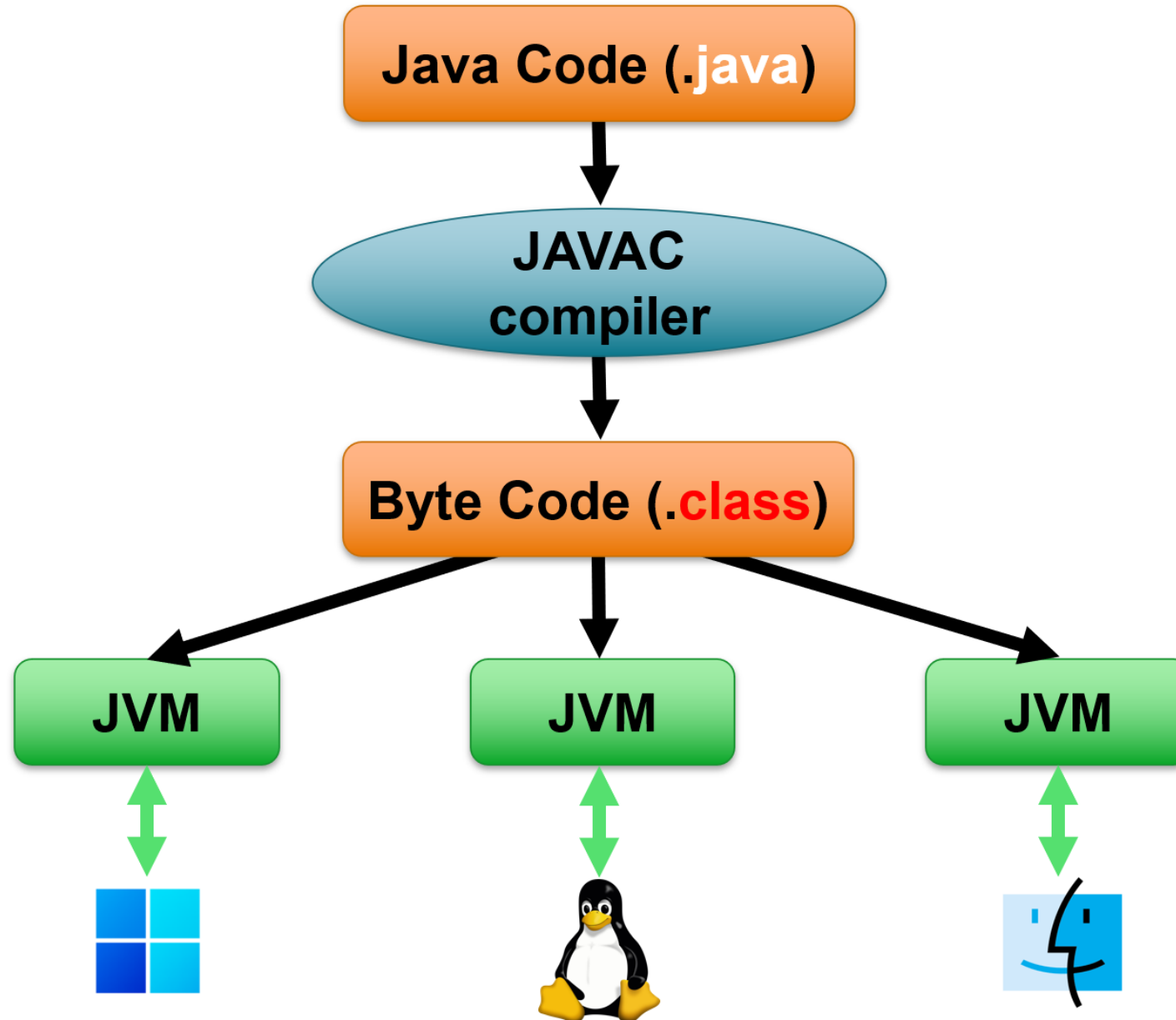
...

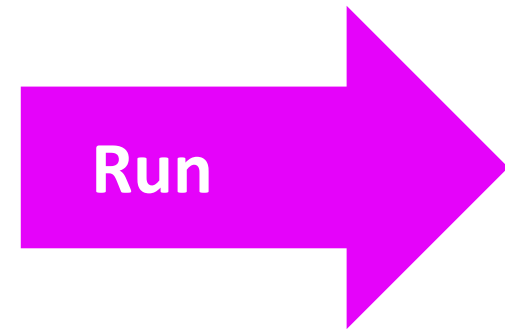
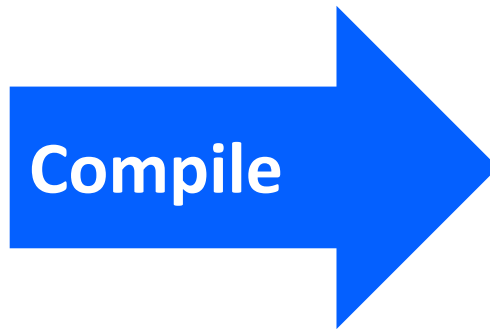


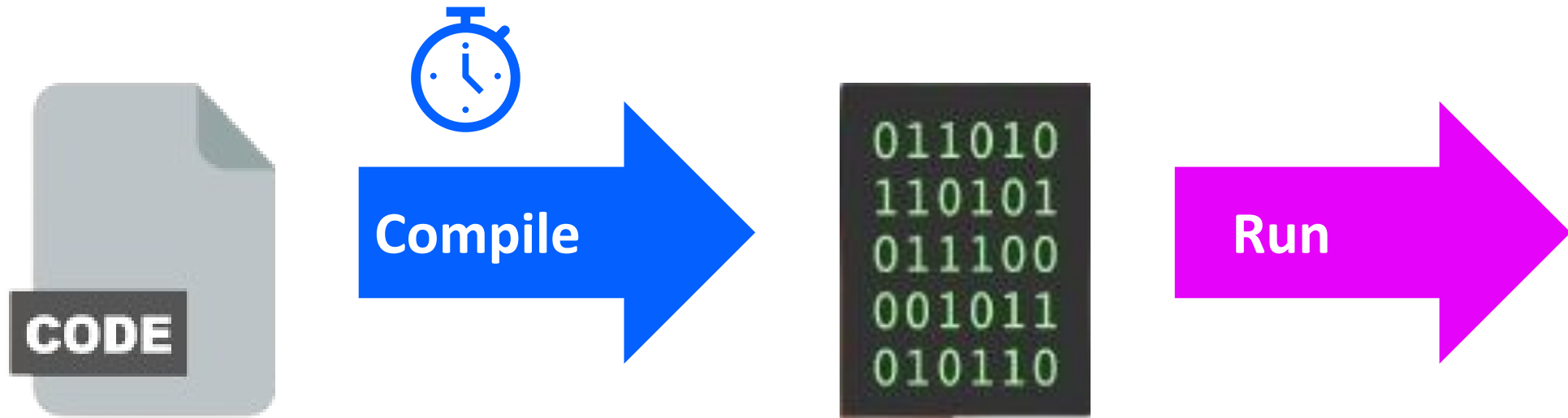
Java Virtual Machine

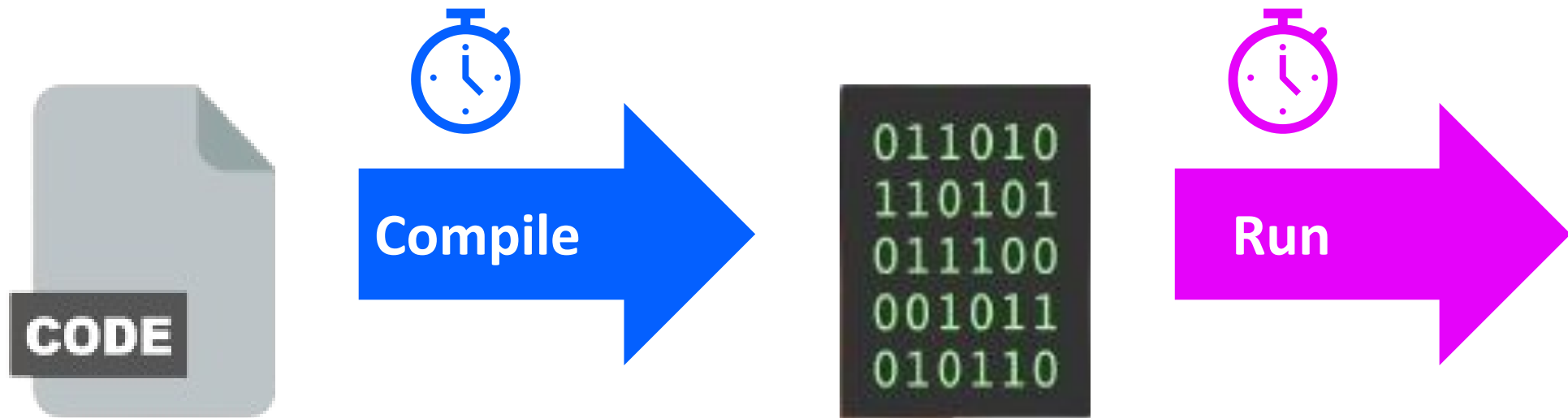


https://youtube.com/shorts/kpe_0FEeyYM









Exception Handling

```
1 import java.io.*;
2
3 public class throws1 {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6     }
7 }
8
9
10
11
12
13
14
15
16
17
18
19
```

Exception Handling

```
1 import java.io.*;
2
3 public class throws1 {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6     }
7 }
8
9
10
11
12
13
14
15
16
17
18
19
```



Exception Handling

```
1 import java.io.*;
2
3 public class throws1 {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt"); 2
6     }
7 }
8
9
10
11
12
13
14
15
16
17
18
19
```

Exception Handling

```
1 import java.io.*;  
2  
3 public class throws1 {  
4     public static void main  
5         FileReader file =  
6     }  
7 }  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19
```



Exception Handling

```
1 import java.io.*;
2
3 public class throws1 {
4     public static void main
5         FileReader file =
6     }
7 }
```



Exception in thread "main" java.io.FileNotFoundException Create breakpoint : test.txt
at java.base/java.io.FileInputStream.open0(Native Method)
at java.base/java.io.FileInputStream.open(FileInputStream.java:213)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:152)
at java.base/java.io.FileInputStream.<init>(FileInputStream.java:106)
at java.base/java.io.FileReader.<init>(FileReader.java:60)
at Exceptions.throws1.main(throws1.java:9)

Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
12
13
14
15
16
17
18
19
```

Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
12
13
14
15
16
17
18
19
```



Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt"); 2
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
12
13
14
15
16
17
18
19
```



Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
12
13
14
15
16
17
18
19
```



Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main
5         try {
6             FileReader fi
7         } catch (IOException
8             System.out.pr
9         }
10     }
11 }
12
13
14
15
16
17
18
19
```



Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
12
13
14
15
16
17
18
19
```



Exception Handling: try-catch

```
1 import java.io.*;
2
3 public class throws2 {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11 }
```

```
An error occurred: test.txt (The system cannot find the file specified)
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```

Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



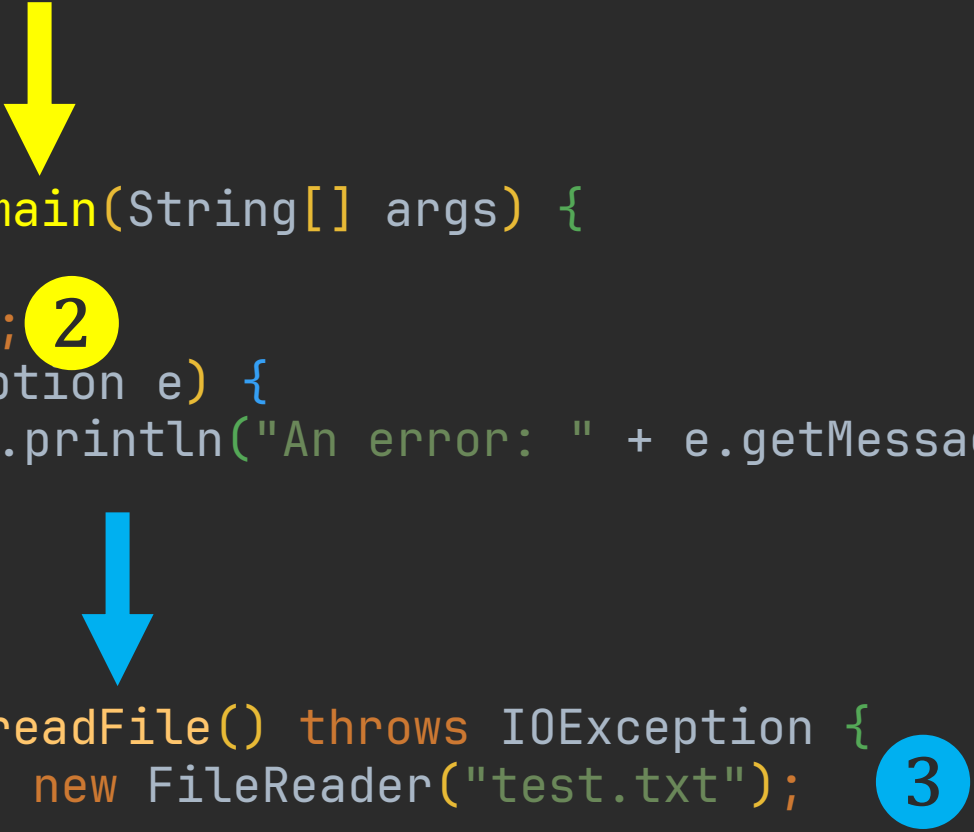
Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile(); 2
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt"); 3
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage()); 4
9         }
10    }
11
12    public static void readFile() throws IOException {
13        FileReader file = new FileReader("test.txt");
14    }
15 }
```



Exception Handling: throws

```
1 import java.io.*;
2
3 public class throws3 {
4     public static void main(String[] args) {
5         try {
6             readFile();
7         } catch (IOException e) {
8             System.out.println("An error: " + e.getMessage());
9         }
10
11         An error occurred: test.txt (The system cannot find the file specified)
12
13     public static void readFile() throws IOException {
14         FileReader file = new FileReader("test.txt");
15     }
16
17
18
19
```



Exception Handling

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6         BufferedReader br = new BufferedReader(file);
7         System.out.println(br.readLine());
8     }
9 }
```

Exception Handling

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6         BufferedReader br = new BufferedReader(file);
7         System.out.println(br.readLine());
8     }
9 }
```



Exception Handling

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6         BufferedReader br = new BufferedReader(file);
7         System.out.println(br.readLine());
8     }
9 }
```



Exception Handling

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         FileReader file = new FileReader("test.txt");
6         BufferedReader br = new BufferedReader(file);
7         System.out.println(br.readLine());
8     }
9 }
```



Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```

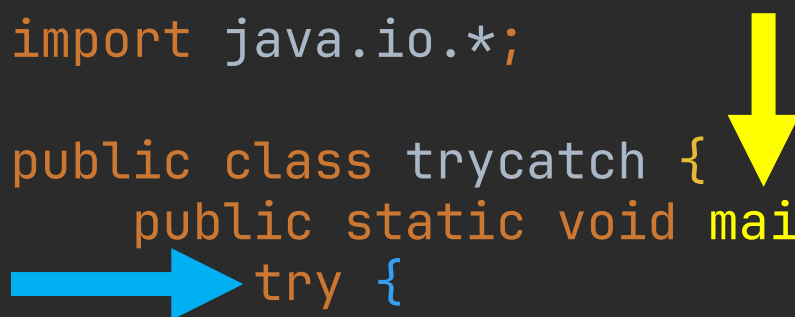
Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



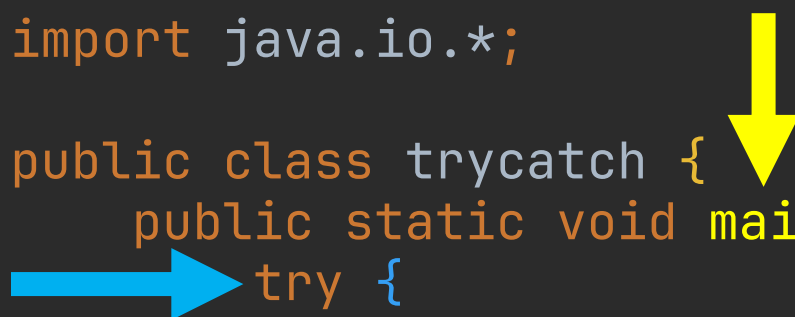
Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



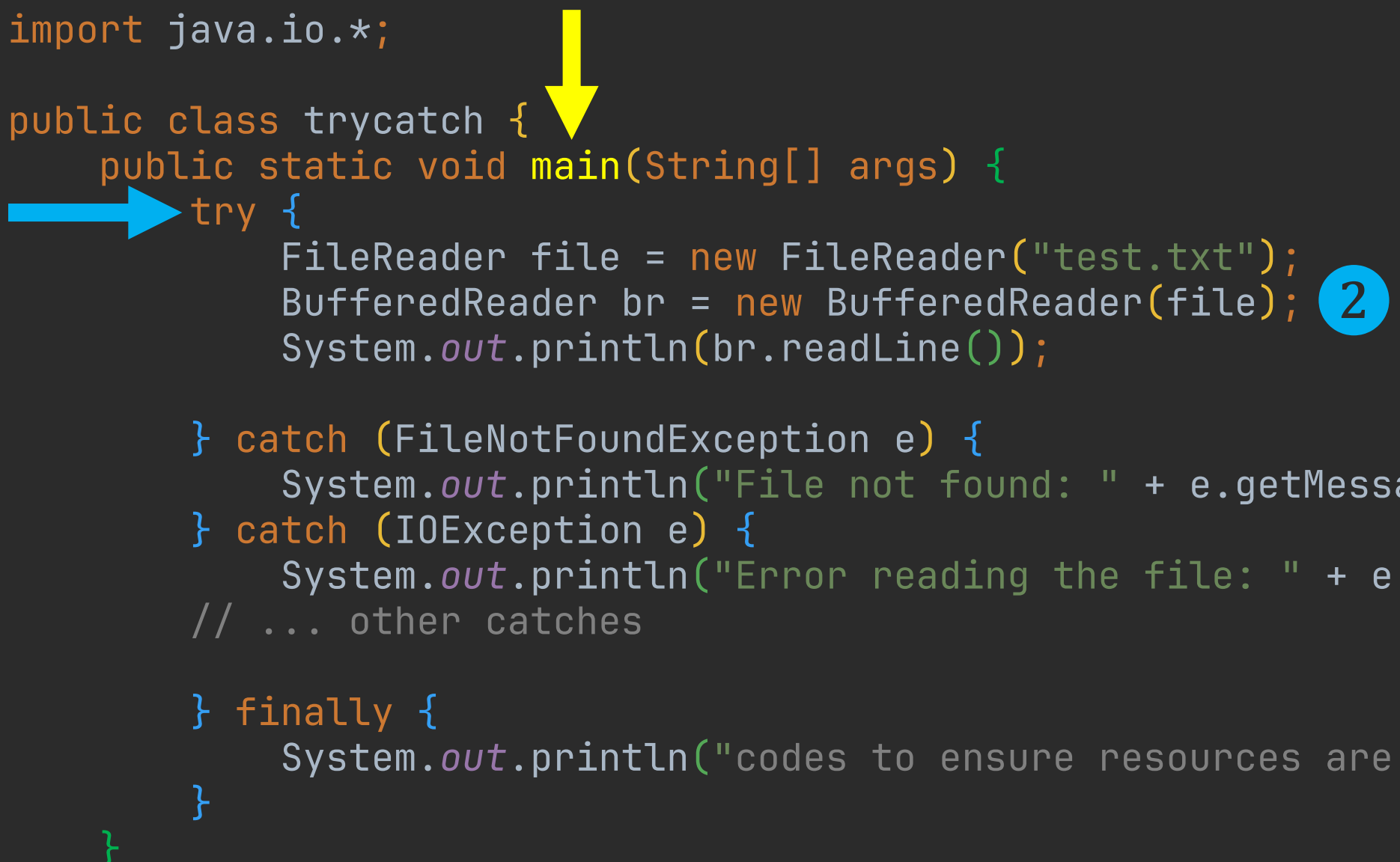
Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



Exception Handling: try-catch-finally

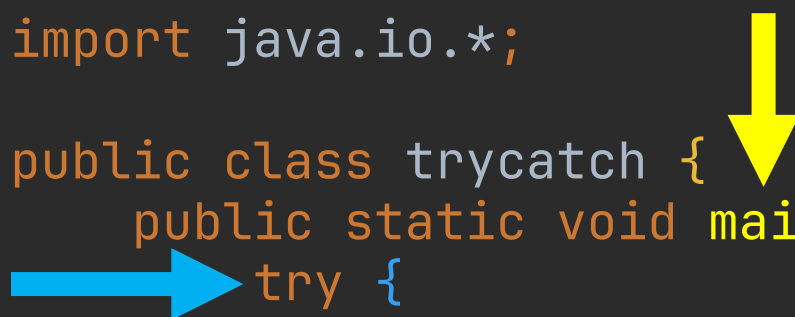
```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



The diagram illustrates the structure of the try-catch-finally block in the provided code. A yellow arrow points down to the opening curly brace of the `main` method on line 4. A blue arrow points right to the `try` keyword on line 5. A blue circle with the number '2' is placed next to the `new BufferedReader(file)` line (line 7), indicating a resource that should be closed in the finally block.

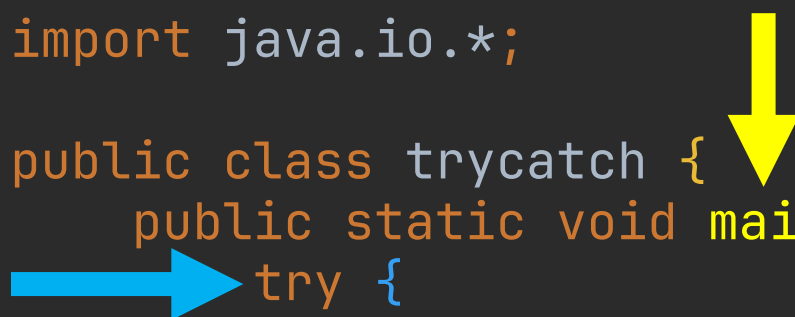
Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14            // ... other catches
15
16        } finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```



Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        }
15
16        finally {
17            System.out.println("codes to ensure resources are closed");
18        }
19    }
}
```

Annotations:

- A yellow arrow points to the `main` method.
- A red warning triangle is placed next to the `FileReader` and `BufferedReader` lines.
- A blue arrow points to the `catch (IOException e)` block.
- A blue circle with the number 3 is next to the `catch (IOException e)` block.
- The output line `Error reading the file: test.txt (No such file or directory)` is highlighted with a yellow box.

Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16    } finally {
17        System.out.println("codes to ensure resources are closed");
18    }
19 }
```



Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16    } finally {
17        System.out.println("codes to ensure resources are closed");
18    }
19 }
```

Annotations:

- A yellow arrow points to the opening curly brace of the `try` block on line 5.
- A red warning triangle with an exclamation mark is positioned next to the `FileReader` and `BufferedReader` instantiation on lines 6 and 7.
- A blue arrow points to the opening curly brace of the `finally` block on line 16.
- A yellow box highlights the text `Ensure resources are closed` on line 18.
- A blue circle with the number `4` is located at the end of line 17.

Exception Handling: try-catch-finally

```
1 import java.io.*;
2
3 public class trycatch {
4     public static void main(String[] args) {
5         try {
6             FileReader file = new FileReader("test.txt");
7             BufferedReader br = new BufferedReader(file);
8             System.out.println(br.readLine());
9
10        } catch (FileNotFoundException e) {
11            System.out.println("File not found: " + e.getMessage());
12        } catch (IOException e) {
13            System.out.println("Error reading the file: " + e.getMessage());
14        } // ... other catches
15
16    } finally {
17        System.out.println("codes to ensure resources are closed");
18    }
19 }
```

Annotations:

- A yellow arrow points to the opening curly brace of the `try` block on line 5.
- A red warning triangle with an exclamation mark is positioned next to the `FileReader` and `BufferedReader` instantiation on lines 6 and 7.
- A green thumbs-up icon is positioned next to the `finally` block on line 16.
- A blue arrow points to the opening curly brace of the `finally` block on line 16.
- A yellow box highlights the text `Ensure resources are closed` on line 18.
- A blue circle with the number `4` is located at the end of line 17.